ACGAIL: Imitation Learning About Multiple Intentions with Auxiliary Classifier GANs

Jiahao Lin and Zongzhang Zhang(✉)

School of Computer Science and Technology,
Soochow University, Suzhou, People's Republic of China
zzzhang@suda.edu.cn

Abstract. As an important solution to decision-making problems, imitation learning learns expert behavior from example demonstrations provided by experts, without the necessity of a predefined reward function as in reinforcement learning. Traditionally, imitation learning assumes that demonstrations are generated from single latent expert intention. One promising method in this line is generative adversarial imitation learning (GAIL), designed to work in large environments. It can be thought as a model-free imitation learning built on top of generative adversarial networks (GANs). However, GAIL fails to learn well when handling expert demonstrations under multiple intentions, which can be labeled by latent intentions. In this paper, we propose to add an auxiliary classifier model to GAIL, from which we derive a novel variant of GAIL, named ACGAIL, allowing label conditioning in imitation learning about multiple intentions. Experimental results on several MuJoCo tasks indicate that ACGAIL can achieve significant performance improvements over existing methods, e.g., GAIL and InfoGAIL, when dealing with label-conditional imitation learning about multiple intentions.

Keywords: Imitation learning · Generative adversarial networks
Multiple intentions · Auxiliary classifier

1 Introduction

To deal with sequential decision-making problems modeled as Markov decision processes (MDPs), reinforcement learning (RL) requires the access to a well-defined reward function or reinforcement signal to find an optimal behavior [19]. Deep RL has achieved remarkable successes in many complex and high-dimensional tasks recently. However, in many complex situations, an appropriate reward function is difficult to be predefined [8]. As a potential alternative solution, imitation learning has no requirement about the access to any knowledge of reward signals and learns to behave by imitating expert demonstrations.

This work was in part supported by National Natural Science Foundation of China (61502323) and High School Natural Foundation of Jiangsu (16KJB520041).

X. Geng and B.-H. Kang (Eds.): PRICAI 2018, LNAI 11012, pp. 321–334, 2018.
https://doi.org/10.1007/978-3-319-97304-3_25

Most of imitation learning algorithms traditionally assume that the demonstrations come from an expert under single latent intention. Among them, generative adversarial imitation learning (GAIL) [12] is state of the art. It is sample efficient in terms of expert data and can scale well to relatively high dimensional environments. Its key idea is to use generative adversarial networks (GANs) [9] to match distributions of states and actions defining expert behavior.

However, in real-world scenarios, we often need to deal with imitation tasks with multiple expert intentions, where single-intention imitation learning algorithms such as GAIL are no longer suitable for learning. They are obsessed with matching the state-action distributions demonstrated by experts blindly, while ignoring latent multiple intentions that lead to difference in performance.

This paper aims to develop an algorithm for imitation learning about multiple intentions in complicated and high-dimensional learning tasks. To our knowledge, the most similar existing work is InfoGAIL [14], which can not only imitate complex behaviors, but also learn interpretable and meaningful representations of complex behavioral data without supervision. This paper takes a different way. We are not willing to apply imitation learning about multiple intentions in an unsupervised way, which can lead to an interpretation beyond understanding easily. We prefer that a single agent can simultaneously employ accurate label-conditional imitation learning about multiple intentions. As a motivating example, imagine that parents as a couple are teaching their kid to drive, the mom demonstrates in a slow fashion with the intention to be careful, while the dad demonstrates in a fast way with the intention to be efficient. Provided the different actions in particular situations, e.g. the dad chooses second gear to accelerate while the mom slows down elegantly to give way to pedestrians, the kid is required to learn from the mixture of demonstrations and imitate parents to cohere with their different intentions simultaneously.

To achieve this goal, we propose a new algorithm, called auxiliary classifier GAIL (ACGAIL), by combining ideas from GAIL and auxiliary classifier GAN [16] (in Sect. 3). ACGAIL uses an auxiliary classifier in GAIL, along with a novel adversarial game, to learn from multiple intentions. The auxiliary classifier plays a role of assisting the adversary which is known as adversarial networks to reconstruct the side information about the multiple latent intentions. This is followed by a closer look at the relationship between ACGAIL and InfoGAIL [14]. Experimental results on several MuJoCo tasks (in Sect. 4) show that ACGAIL can reconstruct the side information and perform label-conditional imitation learning provided by the labeled demonstrations under multiple intentions. ACGAIL can also appear better performance than existing methods, e.g., GAIL and InfoGAIL, in these label-conditional imitation learning scenarios.

2 Background

This section briefly introduces the notations and basic definitions in MDPs, and single-intention/multiple-intention imitation learning methods.

2.1 MDPs

An MDP can be defined by a tuple $(\mathcal{S}, \mathcal{A}, P, r, \rho_0, \gamma)$. In it $\mathcal{S}$ is the state space; $\mathcal{A}$ is the action space; $P : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \to [0, 1]$ is the state transition probability distribution, where $P(s' \mid s, a)$ means the probability over state s' after the agent takes action a in state s; $r : \mathcal{S} \times \mathcal{A} \to \mathbb{R}$ is the reward function, where $r(s, a)$ means the reward obtained after taking action a in state s; $\rho_0 : \mathcal{S} \to [0, 1]$ is the distribution of the initial state s_0; $\gamma \in (0, 1)$ is the discount factor which balances the immediate and delayed rewards. This paper focuses on dealing with the tasks that have continuous state and action spaces. We define $\pi : \mathcal{S} \times \mathcal{A} \to [0, 1]$ as a stochastic policy and η as the expected cumulative discounted reward of π:

$$\eta(\pi) = \mathbb{E}_{s_0, a_0, \ldots}\left[\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t)\right] \tag{1}$$

where $s_0 \sim \rho_0(s_0)$, $a_t \sim \pi(a_t \mid s_t)$, and $s_{t+1} \sim P(s_{t+1} \mid s_t, a_t)$. Let ρ_π be the discount visitation frequencies, i.e., $\rho_\pi(s) = P(s_0 = s \mid \pi) + \gamma P(s_1 = s \mid \pi) + \gamma^2 P(s_2 = s \mid \pi) + \ldots$, where $s_0 \sim \rho_0$ and the actions are taken by π. We can rewrite Eq. 1 in a sum over states rather than time steps: $\eta(\pi) = \sum_{t=0}^{\infty} \int_{s \in \mathcal{S}} P(s_t = s \mid \pi) \int_{a \in \mathcal{A}} \pi(a \mid s) \gamma^t r(s, a) = \int_{s \in \mathcal{S}} \rho_\pi(s) \int_{a \in \mathcal{A}} \pi(a \mid s) r(s, a)$.

2.2 Single-Intention Imitation Learning

Single-intention imitation learning addresses the task of learning a policy from the behavior of an expert driven by one intention, without any access to an explicit reinforcement signal. It mainly includes the following three categories:

Behavioral Cloning [17] learns a policy over state-action pairs in a supervised learning way. One recent work is an end-to-end system that uses a convolutional neural network (CNN) to represent a policy and learns it by behavioral cloning with raw images as inputs [4]. Due to compounding error caused by covariate shift, behavioral cloning suffers from poor generalization.

Inverse Reinforcement Learning (IRL) [1,15] considers that the expert policy is learned under an unknown reward function. Thus, compounding error, a challenging problem for methods that fit single time-step decisions, is no longer a problem for IRL. It learns a policy by iteratively executing the following two steps: it recovers the unknown reward function using expert demonstrations; and it (approximately) solves the RL problem with the learned reward function. However, IRL has the so-called degeneracy issue, i.e., there exist many reward functions making the observed policy optimal. The issue can be eliminated by introducing a casual entropy regularization to the optimization objective, which encourages the algorithm to find a reward function to maximize the casual entropy of the policy [21,22]. Due to the high computational complexity of solving the inner RL problem inside one learning loop, IRL methods are usually inefficient in addressing relatively high-dimensional learning problems.

Generative Adversarial Imitation Learning [9] is a recent imitation learning method inspired by GANs which have achieved prominent successes in the

field of image processing. GANs introduce a two-player minmax game, where a generator captures a data distribution and a discriminator estimates the probabilities whether samples came from the example data. In the training process, the generator tries to confuse the discriminator and the discriminator tries to achieve an accurate estimation. As the discriminator guides the generator to recover the distributions of example data, the discriminator is unable to differentiate between the two distributions eventually. To avoid the inner RL process in IRL, GAIL directly extracts a policy from expert demonstrations based on GANs.

In GAIL, the policy plays a generator role which makes discriminator confusing the state-action pair samples from either the generator or the expert, while the reward function serves as the discriminator trying to distinguish the samples accurately. By training, the reward function guides the policy to match the generated state-action distribution with the expert distribution. The formal GAIL minmax game is denoted as $\min_\theta \max_\omega V(\theta, \omega)$, where $V(\theta, \omega)$ is:

$$\mathbb{E}_{(s,a)\sim\pi_\theta}\left[\log(D_\omega(s,a)\right] + \mathbb{E}_{(s,a)\sim\pi_E}\left[\log(1 - D_\omega(s,a)\right] - \lambda_H H(\pi_\theta) \tag{2}$$

Here, the policy π_θ serves as the generator and the reward function D_ω serves as the discriminator. Both of them are represented as deep neural networks, with parameters θ and ω respectively. The causal entropy of the policy π, denoted as $H(\pi)$, is used as a regularization term together with hyper-parameter λ_H. The policy π_θ can be trained by policy gradient methods such as trust region policy optimization (TRPO) [18], while the discriminator can be optimized using ADAM [13]. By taking advantage of deep neural networks and avoiding the weakness of IRL, GAIL can perform well in relatively high-dimensional problems.

2.3 Multiple-Intention Imitation Learning

There are some existing works that closely relate to imitation learning about multiple intentions. Babes et al. [3] proposed to cluster unlabeled demonstrations by the expectation maximization method. Dimitrakakis and Rothkopf [7] proposed a multi-task learning approach, generalizing the Bayesian approach to IRL. Choi and Kim [6] presented a nonparametric Bayesian approach to IRL by integrating the Dirichlet process mixture model into Bayesian IRL. The algorithms introduced above are based on IRL. However, these IRL-based algorithms require high computational cost of solving RL problems inside the learning loops. Focusing on solving high-dimensional problems, there are some works that extend the GAIL framework, such as InfoGAIL [14] and multi-modal imitation learning [11]. These two extensions learn a single policy that can imitate skills from unlabeled and unstructured demonstrations by maximizing the mutual information between the latent variables and the observed state-action pairs. As shown detailedly in the next section, this paper focuses on learning from labeled demonstrations with multiple intentions instead of unlabeled demonstrations. The most similar work to ours is ACGAN [16], which employs label conditioning that results in image samples exhibiting global coherence by a novel variant of GANs.

3 ACGAIL: Imitation Learning with Auxiliary Classifier

In this section, we first describe the ACGAIL algorithm and its detailed implementation. Then, we take a closer look at its relation to InfoGAIL.

3.1 ACGAIL

In many real-world situations, due to differences of individuals or various purposes, the experts can behave with multiple intentions. So, the example demonstrations provided by experts are mixed with multiple expert intentions. As is often the case, each demonstration can be labeled by a particular intention. Therefore, we can get a mixed set of demonstrations that are labeled by multiple expert intentions. In this paper, we propose that the agent learns from multiple expert intentions simultaneously using one policy. Formally, in this case, we assume that there exists a finite set of K intentions. It can be represented as follows: $\boldsymbol{c_K} = \{c_1, \ldots, c_K\}$. Let us define the prior distribution of intentions as $c \sim p(c)$. For convenience, we simplify the expert policy that includes multiple intentions as π_E, i.e., $\pi_E = \{\pi_{E_{c_1}}, \ldots, \pi_{E_{c_K}}\}$. Instead of learning a policy solely based on the state, the policy can be extended to include a dependence on the intention c as $\pi(a|s, c)$, where c is sampled from the prior $p(c)$.

To reconstruct the side information and perform imitation learning about multiple intentions, we add more structure to the latent space in GAIL along with a specialized optimization objective following the theory of ACGAN. Specifically, we add an auxiliary classifier to adversary which is now composed of two models: an auxiliary classifier defined as C_ψ with parameters ψ and a discriminator defined as D_ω with parameters ω. Furthermore, the policy playing the role of a generator is defined as π_θ with parameters θ. They are all represented by deep neural networks. Working similarly to the discriminator in GAIL, the discriminator in ACGAIL outputs probabilities whether the samples of state-action pairs come from the expert or not. The auxiliary classifier model we added can classify the latent intention labels of state action pairs and output the softmax cross entropy between the latent label variables and logits of classification. In other words, the output of the auxiliary classifier can be interpreted as the probability error of classification. Moreover, the latent intention label variables can be represented as one-hot vectors. The auxiliary classifier assists the adversary reconstructing the side information over latent intention labels. Therefore, we call this new method as auxiliary classifier GAIL. In it, the games for the discriminator and the classifier are given respectively as follows:

$$\min_{\pi_\theta} \max_{D_\omega} \mathbb{E}_{\pi_\theta}[\log(D_\omega(s,a))] + \mathbb{E}_{\pi_E}[\log(1 - D_\omega(s,a))] - \lambda_H H(\pi_\theta) \tag{3}$$

$$\max_{\pi_\theta, C_\psi} \mathbb{E}_{\pi_\theta}[\log(C_\psi(c|s,a))] + \mathbb{E}_{\pi_E}[C_\psi(c|s,a))] \tag{4}$$

where we simplify the $\mathbb{E}_{c\sim p(c),(s,a)\sim\pi(a|s,c)}[\cdot]$ as $\mathbb{E}_{\pi}[\cdot]$. As a result, we can acquire the corresponding objectives for the discriminator and the classifier:

$$L_D = \max_{D_\omega} \mathbb{E}_{\pi_\theta}[\log(D_\omega(s,a))] + \mathbb{E}_{\pi_E}[\log(1 - D_\omega(s,a))] \tag{5}$$

$$L_C = \max_{C_\psi} \mathbb{E}_{\pi_\theta}[\log(C_\psi(c|s,a))] + \mathbb{E}_{\pi_E}[\log(C_\psi(c|s,a))] \tag{6}$$

In our method, the objective of adversary is now composed of the functions from both the discriminator and the classifier:

$$L_{adversary} = L_D + L_C \tag{7}$$

The adversary is trained to maximize the novel optimization objective $L_{adversary}$. It can be considered that the adversary tries to make both the differentiation of the discriminator over sample sources and the classification of the classifier over intention label variables to be more accurate.

By combining the Eqs. 3 and 4 with respect to generator π_θ, we can obtain the optimization objective for the generator π_θ as follows:

$$L_{\pi_\theta} = \min_{\pi_\theta} \mathbb{E}_{\pi_\theta}[\log(D_\omega(s,a))] - \lambda_C \mathbb{E}_{\pi_\theta}[\log(C_\psi(c|s,a))] - \lambda_H H(\pi_\theta) \tag{8}$$

where hyper-parameter λ_C is used to trade off the influence of the discriminator and the classifier to the generator. The discriminator and the classifier can be jointly interpreted as a reward function providing learning signals to the policy. The detailed reward function provided by the adversary can be represented as $-\log(D_\omega(s,a)) + \lambda_C \log(C_\psi(c|s,a))$. It can be thought that the discriminator tries to make the distribution of samples generated by the generator close to the expert distribution. Meanwhile, the auxiliary classifier encourages the policy generated from the generator to match the latent intention label. In such a combination, the adversary guides the policy to confuse the discriminator as well as to reduce the classification error of the classifier, resulting in a policy employing label-conditional imitation learning about multiple intentions.

To sum up, the total minmax game between the adversary and the generator in ACGAIL can be summarized as Eq. 9:

$$\begin{aligned} \min_{\pi_\theta, C_\psi} \max_{D_\omega} \quad & \mathbb{E}_{\pi_\theta}[\log(D_\omega(s,a))] + \mathbb{E}_{\pi_E}[\log(1 - D_\omega(s,a))] \\ & - \mathbb{E}_{\pi_\theta}[C_\psi(c|s,a))] - \lambda_C \mathbb{E}_{\pi_E}[C_\psi(c|s,a))] - \lambda_H H(\pi_\theta) \end{aligned} \tag{9}$$

3.2 Algorithm Implementation

ACGAIL needs to update four networks basically: the discriminator network D_ω, the classifier network C_ψ, the policy network π_θ and a value function working as a baseline. We use TRPO to update π_θ to ensure a monotonic improvement of policy optimization and generalized advantage estimation to reduce the variance of policy gradient estimates. We use ADAM to update both D_ω and

Algorithm 1. ACGAIL

Input: Expert demonstrations with multiple intentions $T_E \sim \pi_E$; initial policy, discriminator, classifier parameters $\theta_0, \omega_0, \psi_0$; initial batch size B; initial the number of steps to apply to adversary M, the number of step to apply to the generator N.

Output: Learned policy π_θ.

for $i = 0, 1, 2, \ldots$ **do**
  Sample a random intention label $c_i \sim p(c)$;
  for M steps **do**
    Sample trajectories from policy π_θ: $\tau_i \sim \pi_{\theta_i}(c_i)$;
    Roll out samples $\mathcal{X}_i \sim \tau_i$ and $\mathcal{X}_E \sim T_E$ with same batch size B;
    Update the discriminator parameters from ω_i to ω_{i+1} with gradient:

$$\Delta_\omega = \mathbb{E}_{\mathcal{X}_i}[\nabla_\omega \log(D_\omega(s,a))] + \mathbb{E}_{\mathcal{X}_E}[\nabla_\omega \log(1 - D_\omega(s,a))]$$

    Update the classifier parameters from ψ_i to ψ_{i+1} with gradient:

$$\Delta_\psi = \mathbb{E}_{\mathcal{X}_i}[\nabla_\psi \log(C_\psi(c|s,a))] + \mathbb{E}_{\mathcal{X}_E}[\nabla_\psi \log(C_\psi(c|s,a))]$$

  end for
  for N steps **do**
    Sample trajectories from policy π_θ: $\tau_i \sim \pi_{\theta_i}(c_i)$ and roll out samples $\mathcal{X}_i \sim \tau_i$;
    Take a policy step from θ_i to θ_{i+1}, using the TRPO update rule with the following objective:

$$-\mathbb{E}_{\mathcal{X}_i}[\log(D_{\omega_{i+1}}(s,a))] + \lambda_C \mathbb{E}_{\mathcal{X}_i}[\log(C_{\psi_{i+1}}(c|s,a))] - \lambda_H H(\pi_\theta)$$

  end for
end for

C_ψ. Since the classifier and the discriminator form jointly as the adversary and work closely, we introduce a structure that the classifier and the discriminator share their parameters and update jointly according to the optimization objective of the adversary, which is similar to the optimization of the dueling network architecture [20]. Moreover, we consider the training improvements of the discriminator and the classifier to be the same since they are equally important to update. As a result, it is not proper to use the hyper-parameter λ_C to trade off the training step sizes in updating the discriminator and the classifier. However, the influences of the discriminator and the classifier towards the generator should not be the same. Therefore, we use the hyper-parameter λ_C in the reward function to balance the influences properly. The training procedure of ACGAIL is presented in Algorithm 1. In it, the finite-horizon trajectory samples include the label information of intention c_i as $\tau_{c_i} = (s_0, a_0, \ldots, s_h, a_h \mid c_i)$, where h represents the length of trajectory. As a result, the set of expert demonstrations $T_E = \{\tau_1, \ldots, \tau_n\}$ is extended to a mixture of state-action pairs labeled by different intentions as $T_E = \{s_0, a_0, c_0, s_1, a_1, c_1, \ldots\}$. So, the input of Algorithm 1 can be considered as a batch of tuples (s_i, a_i, c_i) selected from T_E randomly.

In the iteration of training, we set the numbers of inner loop steps for both the adversary and the generator, denoted as M and N, to balance the training

step sizes of the adversary and the generator. Our setting follows the trick of training GANs, which suggests the adversary to maintain proper accuracy to prevent the generator from vanishing gradient. To speed up training, we suggest to use the behavioral cloning method to initialize policy in some tasks. Note that there are some works to improve the training, such as Wasserstein GAN [2] and improved WGAN [10]. However, due to we are dealing with an adversary constructed by two models, here we follow the training trick of vanilla GAN.

3.3 Relation to InfoGAIL and Mutual Information

Both ACGAIL and InfoGAIL focus on the learning problem about multiple intentions. InfoGAIL tends to interpret the latent intentions by maximizing the mutual information between the latent intention variables and the state-action pairs generated from the generator, where the maximization of the mutual information can be understood that the samples generated by a policy exhibit higher relevance to the corresponding intention labels. However, due to lack of the access to posterior, InfoGAIL can not maximize the mutual information directly. It tries to optimize the surrogate minmax game as follows:

$$\begin{aligned}\min_{\pi_\theta, Q_\psi} \max_{D_\omega} \; & \mathbb{E}_{\pi_\theta}[\log(D_\omega(s,a))] + \mathbb{E}_{\pi_E}[\log(1 - D_\omega(s,a))] \\ & - \lambda_Q \mathbb{E}_{\pi_\theta}[Q_\psi(c|s,a))] - \lambda_H H(\pi_\theta)\end{aligned} \tag{10}$$

The corresponding objectives of the discriminator D_ω, the posterior Q_ψ, and the generator π_θ are:

$$L_{D_\omega} = \max_{D_\omega} \mathbb{E}_{\pi_\theta}[\log(D_\omega(s,a))] + \mathbb{E}_{\pi_E}[\log(1 - D_\omega(s,a))] \tag{11}$$

$$L_{Q_\psi} = \max_{Q_\psi} \mathbb{E}_{\pi_\theta}[\log(Q_\psi(c|s,a))] \tag{12}$$

$$L_{\pi_\theta} = \min_{\pi_\theta} \mathbb{E}_{\pi_\theta}[\log(D_\omega(s,a))] - \lambda_Q \mathbb{E}_{\pi_\theta}[\log(Q_\psi(c|s,a))] - \lambda_H H(\pi_\theta) \tag{13}$$

Here, Eq. 11 is similar to Eq. 5 in guiding the generator to imitate the expert behavior. Equation 13, similar to Eq. 8, is the objective for π_θ. By comparing them, we find that ACGAIL can also be interpreted as trying to maximize the mutual information between the latent label variables and the state-action pairs.

The difference arises from Eq. 12 compared to Eq. 6. Following the GAN fashion, the training of the auxiliary classifier in ACGAIL is actually fed either labeled expert samples or generated samples. Leveraging the training through labeled expert samples, the auxiliary classifier can acquire the access to a relatively accurate posterior of the state-action pairs. Thus, ACGAIL maximizes the mutual information directly rather than the surrogate objective. Thus, the auxiliary classifier introduced in ACGAIL is able to reconstruct the side information about latent intention labels, which results in the policy employing label-conditional imitation learning in coherence with corresponding expert intentions. Meanwhile, dealing with unlabeled demonstrations, InfoGAIL finds a set of most significant latent representations as intentions and learns from it in an unsupervised way. Notice that ACGAIL is training with labeled demonstrations while

Table 1. Parameters of tasks and expert's average performance

Task	State space	Action space	Expert label 0	Expert label 1
Hopper-v1	11	3	3589.09	672.55
Walker2d-v1	17	6	3550.94	725.99
HalfCheetah-v1	17	6	3889.23	1487.27
Humanoid-v1	376	117	2267.36	488.31

InfoGAIL is training with unlabeled demonstrations. The difference of preconditions about demonstrations leads to the prominent difference in performance. Compared to InfoGAIL which can result in an interpretation beyond understanding easily, the advantage of ACGAIL is that the auxiliary classifier assists the algorithm to achieve accurate label-conditional imitation learning about multiple intentions provided with labeled demonstrations.

4 Experiments

This section aims to answer the following questions: (1) Can ACGAIL use only one policy to employ label-conditional imitation learning about multiple expert intentions? (2) Can ACGAIL scale well to high-dimensional tasks?

4.1 Environmental Setup

We evaluate Algorithm 1 against two baselines, GAIL and InfoGAIL, in a series of challenging high-dimensional simulated robotic tasks in MuJoCo. Each task comes with a true reward function, defined in the OpenAI Gym [5]. The expert policies with multiple intentions are generated by running TRPO in different numbers of iterations with these true reward functions. Then, we would like to use the expert policies to sample 1,500 trajectories and label the trajectories with the corresponding latent intention variables. The labeled trajectories sampled from expert policies are shuffled as labeled state-action pairs. For conciseness, we generate two latent expert intentions for each task. Table 1 shows the information of tasks and the expert's average performance under each intention. Columns 2–3 list the dimensions of the state space and the action space for each task, and Columns 4–5 list the expert's average performance under latent intentions label variables 0 and 1. Each expert's performance is averaged over the cumulative discounted rewards (a.k.a. returns) of 1,500 expert trajectories.

4.2 Imitation Learning About Multiple Intentions

In our algorithm, we get four neural networks to update: discriminator, auxiliary classifier, policy and value function. The setting of neural networks is the same as in GAIL: 2 hidden layers of 100 units each, with tanh nonlinearities between

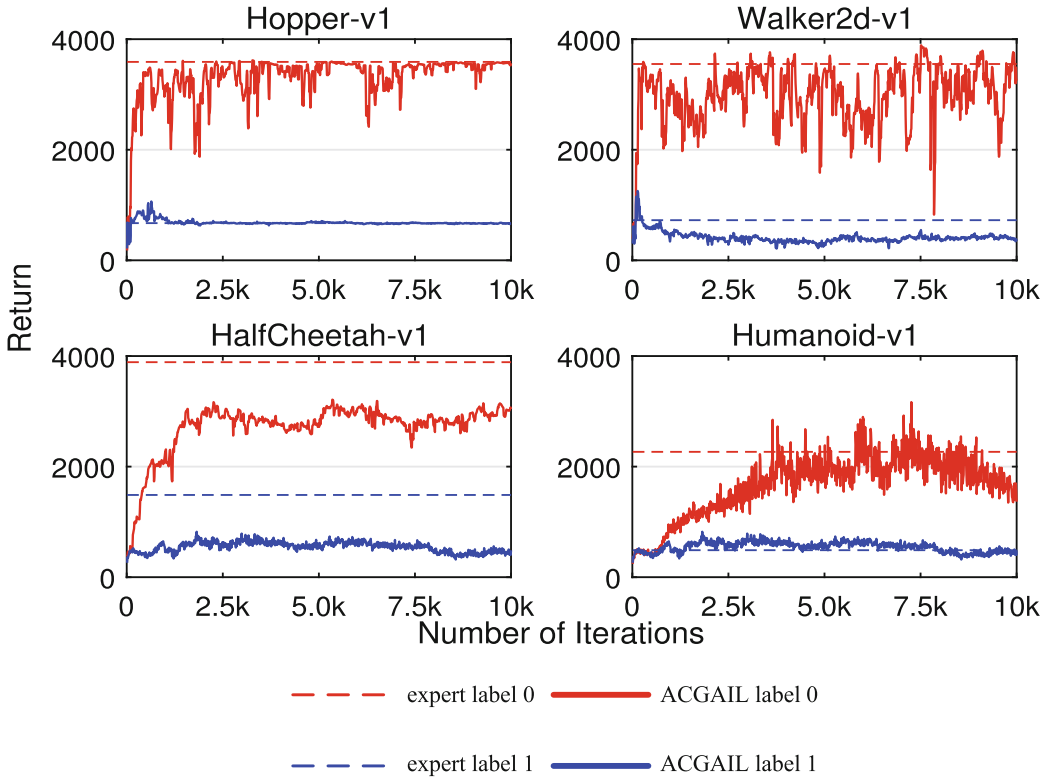

Fig. 1. ACGAIL's results on four tasks. The dashed lines represent the average return of trajectories performed by expert; the bold lines represent the return of one trajectory generated by ACGAIL. The red and blue lines, respectively, represent the results under intention label variables 0 and 1. The horizontal axis shows the number of training iterations, and the vertical axis shows the return averaged over last 40 trajectories representing the performance of learned policy. (Color figure online)

them. The same networks are also used in the two baseline algorithms. As we share the parameters in ACGAIL, the discriminator and the auxiliary classifier are also structured by 2 hidden layers of 100 units each, while the hidden layers are shared and the output layers work differently. We use TRPO and ADAM to train the policy and the value function respectively, and also use ADAM to train the adversary. We set the learning rate of ADAM for the adversary to 5×10^{-5}, a relatively small value, according to the training trick of GANs. We choose to set $\lambda_H = 0$ since we find that the empirical performance of ACGAIL is insensitive to this parameter, and set $\lambda_C = 0.5$.

In Fig. 1, we present the results of running ACGAIL on four different tasks by showing 10,000 training iterations. The results show that, at the beginning of training, the policies under two different intentions are roughly similar; when the training process goes on, the policies under different intentions exhibit discriminating performance, each of which performs closely to the expert behavior under the corresponding intention. On Hopper-v1 and Walker2d-v1, our algorithm begins to employ label-conditional imitation learning after about 200 training iterations, while on HalfCheetah-v1 and Humanoid-v1, the differences begin after around 1,000 training iterations. Furthermore, on Hopper-v1, each of the

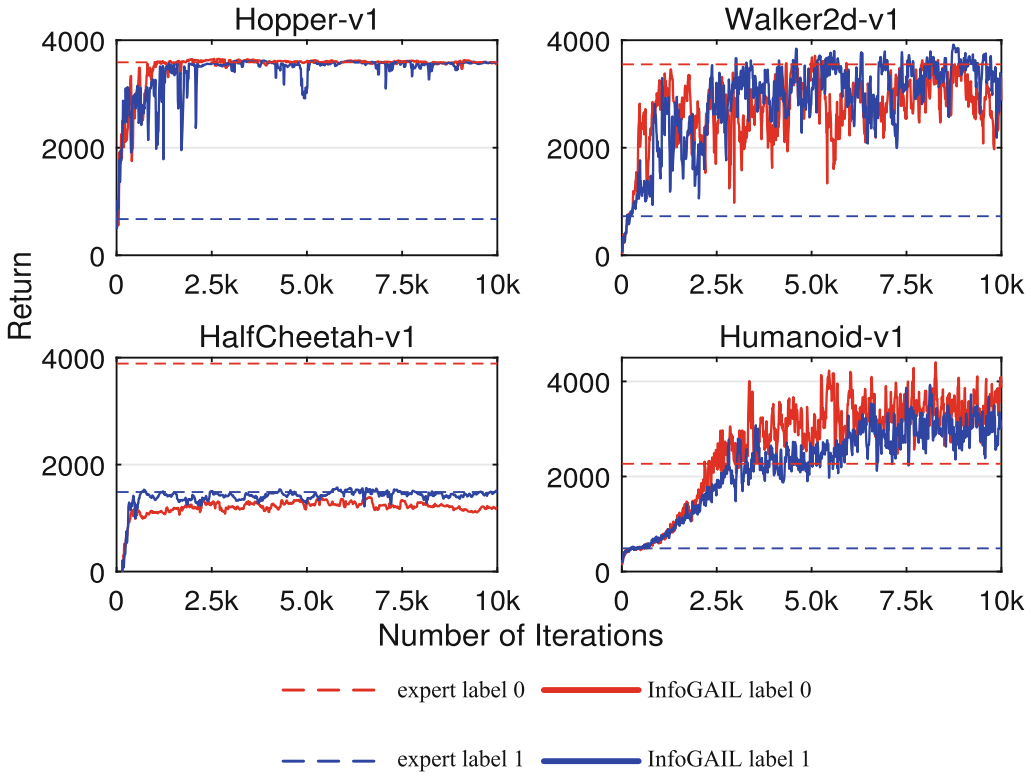

Fig. 2. InfoGAIL's results on four tasks. The dashed lines represent the average return of trajectories performed by expert; the bold lines represent the return of one trajectory generated by generator. The red and blue lines, respectively, represent the results under intention label variables 0 and 1. The horizontal axis shows the number of training iterations, and the vertical axis shows the return averaged over last 40 trajectories representing the performance of learned policy. (Color figure online)

policies conditioned under two different intention labels perfectly converges to the corresponding expert returns. On Walker2d-v1, the return of the policy under intention label 0 exhibits more fluctuation. It can be interpreted that the returns of expert demonstrations exhibit volatility since the dimensions or cardinalities of both state and action spaces become larger. On HalfCheetah-v1, the returns of policies vary and converge to the returns under the corresponding expert intentions. On Humanoid-v1, due to the issue of high dimensionality, the training is quite slow and quite difficult to achieve a perfect convergence. Above all, the results of all tasks show that ACGAIL can exhibit different returns close to the expert's returns under the corresponding latent intentions and scale well to high-dimensional tasks.

In Fig. 2, we demonstrate the results of InfoGAIL on the four MuJoCo tasks. The results of GAIL are listed in Table 2. The setting for InfoGAIL is the same as in ACGAIL. From the results of the four tasks in Figs. 1 and 2, we can see that ACGAIL is able to exhibit different behaviors/performance under different latent intention label variables. Furthermore, ACGAIL is able to generate label-conditional behavior samples that precisely approximate to expert performance under corresponding latent intention labels. While InfoGAIL struggles to

Table 2. Performance errors over all latent intentions

Task	ACGAIL	InfoGAIL	GAIL
Hopper-v1	**0.0097**	2.1340	2.1678
Walker2d-v1	**0.2796**	1.6167	1.7951
HalfCheetah-v1	**0.2921**	0.3642	0.5546
Humanoid-v1	**0.1256**	2.7784	3.0664

interpret the latent intentions ignoring the side information over labeled demonstrations.

We suggest to use the sum of the differences in average returns over all intentions as a quantitative criterion of the similarity between generated state-action samples and expert samples over multiple intentions:

$$X = \sum_{c_i \in \boldsymbol{c_K}} p(c_i) \frac{\left| \bar{R}_{\pi_\theta, c_i} - \bar{R}_{\pi_E, c_i} \right|}{\bar{R}_{\pi_E, c_i}} \tag{14}$$

where $\bar{R}_{\pi_\theta}$ or $\bar{R}_{\pi_E}$ represents the average return of sampled trajectories given a policy π_θ or π_E, and $p(c_i)$ represents the prior probability of intention label variable c_i. Specifically, $p(c_i)$ should be 0.5 since we sample the two intention label variables randomly in our algorithm. The criterion represents performance errors over all latent intentions. It uses absolute values to accept both positive and negative deviations to the expert's performance, and take the expert's performance as the denominator. Its output X can be interpreted as the degree of the performance variance between the expert and the generator over all intentions. It is obvious that a smaller value of X is preferred. For quantifying the differences over all intentions for the algorithms, we use the trajectory samples with training iterations from $8,000$ to $9,000$ for each algorithm. The outputs of the criterion are reported in Table 2.

Table 2 shows that ACGAIL has lower variance over latent intentions than InfoGAIL as it employs label-conditional imitation learning about multiple intentions. Because InfoGAIL tries to interpret the latent intentions, it exhibits lower variance than GAIL which only works properly under single intention.

5 Conclusion and Future Work

In this paper, we introduce auxiliary classifier as an additional model to vanilla GAIL, which is able to use single policy to employ label-conditional imitation learning about multiple intentions. Furthermore, the experimental results show that ACGAIL can better deal with the situations where the demonstrations are labeled by multiple latent expert intentions.

We noticed that there are some drawbacks in the current ACGAIL implementation. Its performance appears worse as the number of latent expert intentions increases. In addition, it is tricky to search proper hyper-parameters of time

steps to improve training for particular tasks. In the future, we will explore ways of eliminating the tricky issue of searching time steps of training. As a matter of fact, we are digging into the possibility of extending the remarkable work of Wasserstein GAN to ACGAIL. Moreover, we are trying to find a more stable structure to achieve imitation learning about multiple intentions, and then use it to learn from several latent intentions.

References

1. Abbeel, P., Ng, A.Y.: Apprenticeship learning via inverse reinforcement learning. In: ICML, pp. 1–8 (2004)
2. Arjovsky, M., Chintala, S., Bottou, L.: Wasserstein generative adversarial networks. In: ICML, pp. 214–223 (2017)
3. Babes, M., Marivate, V., Subramanian, K., Littman, M.L.: Apprenticeship learning about multiple intentions. In: ICML, pp. 897–904 (2011)
4. Bojarski, M., et al.: End to end learning for self-driving cars. arXiv preprint arXiv:1604.07316 (2016)
5. Brockman, G., et al.: OpenAI Gym. arXiv preprint arXiv:1606.01540 (2016)
6. Choi, J., Kim, K.E.: Nonparametric Bayesian inverse reinforcement learning for multiple reward functions. In: NIPS, pp. 305–313 (2012)
7. Dimitrakakis, C., Rothkopf, C.A.: Bayesian multitask inverse reinforcement learning. In: Sanner, S., Hutter, M. (eds.) EWRL 2011. LNCS (LNAI), vol. 7188, pp. 273–284. Springer, Heidelberg (2012). https://doi.org/10.1007/978-3-642-29946-9_27
8. Finn, C., Levine, S., Abbeel, P.: Guided cost learning: deep inverse optimal control via policy optimization. In: ICML, pp. 49–58 (2016)
9. Goodfellow, I., et al.: Generative adversarial nets. In: NIPS, pp. 2672–2680 (2014)
10. Gulrajani, I., Ahmed, F., Arjovsky, M., Dumoulin, V., Courville, A.C.: Improved training of Wasserstein GANs. In: NIPS, pp. 5769–5779 (2017)
11. Hausman, K., Chebotar, Y., Schaal, S., Sukhatme, G., Lim, J.J.: Multi-modal imitation learning from unstructured demonstrations using generative adversarial nets. In: NIPS, pp. 1235–1245 (2017)
12. Ho, J., Ermon, S.: Generative adversarial imitation learning. In: NIPS, pp. 4565–4573 (2016)
13. Kingma, D.P., Ba, J.: ADAM: a method for stochastic optimization. ICLR (2015)
14. Li, Y., Song, J., Ermon, S.: InfoGAIL: interpretable imitation learning from visual demonstrations. In: NIPS, pp. 3815–3825 (2017)
15. Ng, A.Y., Russell, S.J.: Algorithms for inverse reinforcement learning. In: ICML, pp. 663–670 (2000)
16. Odena, A., Olah, C., Shlens, J.: Conditional image synthesis with auxiliary classifier GANs. In: ICML, pp. 2642–2651 (2017)
17. Pomerleau, D.A.: Efficient training of artificial neural networks for autonomous navigation. Neural Comput. **3**(1), 88–97 (1991)
18. Schulman, J., Levine, S., Abbeel, P., Jordan, M., Moritz, P.: Trust region policy optimization. In: ICML, pp. 1889–1897 (2015)
19. Sutton, R.S., Barto, A.G.: Reinforcement Learning: An Introduction. MIT Press, Cambridge (1998)

20. Wang, Z., Schaul, T., Hessel, M., Hasselt, H., Lanctot, M., Freitas, N.: Dueling network architectures for deep reinforcement learning. In: ICML, pp. 1995–2003 (2016)
21. Ziebart, B.D., Bagnell, J.A., Dey, A.K.: Maximum causal entropy correlated equilibria for Markov games. In: AAMAS, pp. 207–214 (2011)
22. Ziebart, B.D., Maas, A.L., Bagnell, J.A., Dey, A.K.: Maximum entropy inverse reinforcement learning. In: AAAI, pp. 1433–1438 (2008)